

Data vs Database vs DBMS

It provides an overview of **Data**, **Database**, and **DBMS (Database Management System)**.

1. Data

- Any information, facts, or figures.
- Data is the basic unit of information. It can be raw, unorganized, and in various forms.

2. Database

- A collection of data that is stored in a secured format and can be easily accessed.
- Database is like a structured storage area for data.
- Databases help in organizing data so that it can be efficiently managed and retrieved when needed.
- Security and easy accessibility are key characteristics of a well-designed database.

3. DBMS (Database Management System)

- A system software responsible for the creation, retrieval, and management of the database.

Key Functions and Roles:

- **Data Management:** DBMS is used to manage databases by ensuring that the data remains consistent and organized.
- **User Interface:** Acts as an interface between the database and the end user, simplifying data interaction.
- **Data Consistency and Organization:** Ensures that data is maintained in a structured manner, making it easier to access and update.
- **Security and Integrity:** Helps protect data against unauthorized access and ensures that data integrity is maintained throughout its lifecycle.

Summary

- **Data** is the raw information or facts.
- A **Database** is an organized collection of data stored securely.
- A **DBMS** is the software that facilitates the creation, management, and retrieval of data in a database.

RDBMS vs DBMS

DBMS (Database Management System)

- A DBMS is a software system that provides an organized way to store, manage, and retrieve data efficiently.
- It ensures that data can be stored in an **organized** manner and is **accessible anytime**.
- DBMS allows CRUD operations (Create, Read, Update, Delete) on databases.

RDBMS (Relational Database Management System)

- RDBMS is an advanced version of DBMS where data is stored in the form of **relations (tables)**.
- Data is maintained using **structured tables** and relationships among them.
- It provides **data integrity, normalization, and relationships** between multiple tables, increasing efficiency.
- Storing data in multiple tables **improves performance and reduces redundancy**.

Example of RDBMS Schema

Example of how data is stored in a relational model:

Student Table (Table 1)

sid	sname	sage	city
1	Rohan	18	1

City Table (Table 2)

id	city-name
1	Pune

Here, **city** in the **Student Table** is a **foreign key** referencing **id** in the **City Table**, forming a relationship between the two tables.

Popular RDBMS Databases

- MySQL
- Oracle
- Microsoft SQL Server

Introduction to SQL and MySQL

What is SQL?

- SQL (Structured Query Language) is the standard language used to interact with RDBMS.
- It is used to **store, retrieve, and manipulate** data in a relational database.
- SQL commands are used to perform operations like inserting, updating, deleting, and querying data.
- Writing SQL commands for any operation is referred to as an **SQL Query**.

What is MySQL?

- MySQL is an open-source Relational Database Management System (RDBMS).
- It is used to store and manage data in a **structured and organized** way.
- MySQL ensures that data is stored **securely** and is **accessible anytime**.
- It allows users to communicate with the database using **SQL queries**.
- MySQL is widely used in web applications, enterprise systems, and cloud databases.

Key Differences between SQL and MySQL

Feature	SQL	MySQL
Definition	A query language for managing data in RDBMS	A database management system that implements SQL
Usage	Used to query and manipulate data	Used to store and manage relational data
Type	Language	Software
Operations	Defines and manipulates data	Stores, processes, and secures data

By using **SQL** commands in **MySQL**, we can efficiently work with databases and perform various operations on stored data.

Database Components

A database system consists of two main components: the **server** and the **client**. These components work together to manage and process data efficiently.

1. Server Component

- The server is where actual data is stored.
- It processes queries received from clients, executes them against the database, and returns the results.
- The server component must be installed and running for the database system to function.

2. Client Component

- The client is a lightweight software where we write SQL queries.
- It must have the address of the server component to establish a connection.
- The database client serves as a user interface or application that interacts with the database server to perform operations such as querying, updating, and managing data.

Types of Client Components

The client component can be categorized into two types:

1. GUI (Graphical User Interface)

- Provides a user-friendly interface with visual tools for interacting with the database.
- More convenient and easier to use.
- Example: MySQL Workbench

2. CLI (Command Line Interface)

- Uses text-based commands to interact with the database.
- Requires knowledge of SQL commands and syntax.
- Provides more flexibility and control over database operations.

Learning with MySQL

- We are using **MySQL** to understand database components.
- We will install both the **client** and **server** on our system to practice database operations.

MySQL Installation Guide for Windows

This guide provides step-by-step instructions for installing **MySQL** on a Windows system, configuring it, and setting up the necessary environment variables.

Step 1: Download MySQL Installer

1. Go to the official **MySQL** website: [MySQL.com](https://www.mysql.com).
2. Navigate to **Downloads** → **MySQL Community (GPL) Downloads**.
3. Click on **MySQL Installer for Windows**.
4. Select the **latest version** available.
5. Choose **Operating System** → **Microsoft Windows**.
6. Click **Download** and wait for the process to complete.

Step 2: Install MySQL on Windows

1. Locate the **downloaded MySQL Installer** and double-click to run it.
2. Choose a **Setup Type**: Select **Custom**.
3. Select **Products**:
 - Add the following products for installation:
 - **MySQL Server**
 - **MySQL Workbench**
 - **MySQL Shell**
4. Click **NEXT** → **NEXT** → **NEXT**.
5. Set a **MySQL Password** for the root user and note it down.
6. In **Server File Permission**, choose Yes, grant full access.
7. Click **NEXT** → **EXECUTE** → Wait for the installation to complete.
8. Click **FINISH** → **FINISH** to exit the installer.

Step 3: Configure Environment Variables for MySQL

To use MySQL from the command line, set up the **environment path variable**:

1. Locate the **MySQL installation directory**:
 - Go to the installed location: `MySQL Server -> Bin`
 - **Copy the full path** (e.g., `C:\Program Files\MySQL\MySQL Server X.X\bin`)
2. Open **Environment Variables**:
 - Navigate to **Advanced** → **Environment Variables**.
 - Under **User Variables**, find Path → Click Edit.
 - Click **New** → Paste the copied MySQL **Bin** path.
 - Click **OK** → **OK** to save changes.

Step 4: Verify MySQL Installation

To confirm MySQL is installed and accessible from the command line:

1. Open **Command Prompt (cmd)**
2. Run the following command:

```
mysql --version
```

If MySQL is installed correctly, you should see an output similar to:

```
mysql Ver 8.0.XX for Win64 on x86_64 (MySQL Community Server - GPL)
```

MySQL Workbench & CLI Client Overview

MySQL provides two main interfaces for database management:

1. **MySQL Workbench** – A graphical user interface (GUI) for managing databases.
2. **MySQL Command Line Interface (CLI)** – A text-based interface to execute SQL commands.

Both interfaces allow users to create, manage, and interact with databases.

MySQL Workbench

Key Features:

- **Default Connection:** MySQL Workbench provides a default connection (**Local Instance MySQL80**).
- **Localhost Connection:** The term `localhost` signifies that MySQL Workbench is connected to the MySQL Server running on the same system.
- **Creating a New Connection:**
 1. Open MySQL Workbench.
 2. Click on **+** (**New Connection**).
 3. Enter a **Connection Name**.
 4. Verify **Hostname** and **Username**.
 5. Store the password securely in **Vault**.
 6. Click **OK** to create the connection.
- **Multiple Connections:** Users can create multiple connections to different MySQL servers.
- **Viewing Available Databases:**

```
SHOW DATABASES;
```

This command lists all the databases available in the server.

MySQL Command Line Interface (CLI)

Key Features:

- The CLI provides a text-based interface for executing MySQL commands.
- All operations that can be performed in MySQL Workbench can also be executed in the CLI.
- **Connecting to MySQL via CLI:**

```
mysql -u root -p
```

This command prompts for a password and connects to MySQL.

- **Viewing Available Databases in CLI:**

```
SHOW DATABASES;
```

- **Selecting a Database:**

```
USE database_name;
```

- **Displaying Tables in a Database:**

```
SHOW TABLES;
```

Both MySQL Workbench and CLI are powerful tools for database management. While the GUI is user-friendly for beginners, the CLI is essential for automation and scripting.

Creating and Deleting Databases

Creating and deleting databases are fundamental operations in SQL. These commands allow users to manage databases efficiently. SQL commands are **case-insensitive**, meaning they can be written in either uppercase or lowercase. However, by convention, SQL commands are usually written in **uppercase** to enhance readability.

1. Creating a Database

To create a new database, use one of the following commands:

```
CREATE DATABASE db_name;  
CREATE SCHEMA db_name;
```

Both commands perform the same function: creating a new database.

Conditional Creation

If you want to create a database **only if it does not already exist**, use:

```
CREATE DATABASE IF NOT EXISTS db_name;
```

This ensures that the database is created only if it is not already present, preventing errors when running the command multiple times.

Example:

```
CREATE SCHEMA telusko_db;  
CREATE DATABASE IF NOT EXISTS telusko_db;
```

2. Deleting a Database

To delete an existing database, use one of the following commands:

```
DROP DATABASE db_name;  
DROP SCHEMA db_name;
```

These commands permanently remove the database and all of its associated data, so they should be used with caution.

Example:

```
DROP DATABASE telusko_db;
```

Important Notes:

- **Dropping a database is irreversible**—ensure you have a backup if necessary.
- Deleting a database will remove all tables and data within it.

Data Types in SQL

Data types define the type of data that can be stored in a table column. Choosing the right data type ensures efficient storage and data integrity.

Common Data Types

1. Numeric Data Types

- **INT**: Used to store non-decimal numeric data. It can hold both positive and negative numbers.

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    age INT  
);
```

- **FLOAT**: Used to store real numbers with decimal values.

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    price FLOAT  
);
```

2. String Data Types

- **CHAR(n)**: Used to store fixed-length text data. The length `n` must be specified.

```
CREATE TABLE users (  
    username CHAR(10)  
);
```

- **VARCHAR(n)**: Used to store variable-length text data, where `n` is the maximum length.

```
CREATE TABLE customers (  
    name VARCHAR(50)  
);
```

3. Date & Time Data Types

- **DATE**: Stores date values in the format `YYYY-MM-DD`.

```
CREATE TABLE events (  
    event_date DATE  
);
```

- **TIME**: Stores time values in the format `HH:MI:SS`.

```
CREATE TABLE shifts (  
    shift_time TIME  
);
```

- **DATETIME**: Stores both date and time values.

```
CREATE TABLE logs (  
    log_time DATETIME  
);
```

4. Boolean Data Type

- **BOOLEAN**: Stores `TRUE` or `FALSE` values.

```
CREATE TABLE users (  
    is_active BOOLEAN  
);
```

Summary

- Use **INT** for whole numbers where decimals are not needed.
- Use **FLOAT** for values requiring decimal precision, like prices.
- Use **CHAR** when all values are of a fixed length, and **VARCHAR** when lengths vary.
- Use **DATE**, **TIME**, or **DATETIME** depending on the specific requirement of storing time-related information.
- Use **BOOLEAN** for storing true/false values efficiently.

By selecting the correct data type, we can optimize storage, improve performance, and maintain data integrity.

Creating Tables in the Database

- While creating a table, it is required need to specify:
 - **Table name**
 - **Column names**
 - **Data types** for each column
- Each column has a specific data type that defines the kind of data it can store.
- Multiple tables can be created within a single database.

Syntax for Creating a Table

```
CREATE TABLE table_name (  
    columnName datatype,  
    columnName datatype  
);
```

Selecting a Database

Before creating tables, you must specify the database you want to use:

```
USE db_name;
```

This ensures that the tables are created in the correct database.

Viewing Table Structure

To view the structure of a table in a database, use:

```
SELECT * FROM db_name.table_name;
```

Example: Creating Tables

Step 1: Use the Database

```
USE telusko_db;
```

Step 2: Create `personalInfo` Table

```
CREATE TABLE personalInfo (  
    id INT,  
    name VARCHAR(40),  
    date_of_birth DATE,  
    city VARCHAR(40)  
);
```

Step 3: Create another **personalEducation** Table

```
CREATE TABLE personalEducation (  
    eid INT,  
    Courses VARCHAR(30),  
    city_of_Education VARCHAR(40)  
);
```

Inserting Data into a Table

1. Single Row Insertion

- A row in a table is referred to as a **record**.
- The `INSERT INTO` statement is used to insert data into a table.
- The syntax for inserting a single record into a table is:

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ..
```

- Example:

```
INSERT INTO personalInfo (id, name, date_of_birth, city)
VALUES (1, 'Rohan', '1990-06-14', 'Bengaluru');
```

- **Note:** Column names are optional in the INSERT statement. However, when omitted, values should be provided in the exact order of the table's column definitions.

```
INSERT INTO personalInfo VALUES (1, 'Rohan', '1990-06-14', 'Bengaluru');
```

Inserting Data into a Table

2. Inserting Multiple Rows

- Multiple records can be inserted into a table using a **single query** using comma-separated values.
- This improves performance as it reduces the number of database calls.
- The syntax for inserting multiple rows at once is:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...),
      (value1, value2, ...),
      (value1, value2, ...);
```

- Example:

```
INSERT INTO personalInfo (id, name, date_of_birth, city)
VALUES (6, 'Ramesh', '1990-04-14', 'Pune'),
      (4, 'Suresh', '1990-04-16', 'Mysuru'),
      (5, 'Ramesh', '1990-04-15', 'Mumbai');
```


Primary Key Constraint in SQL

The Primary Key constraint uniquely identifies each record in a table. It ensures that no duplicate or null values exist in the designated column.

Key Features:

- A Primary Key provides a unique identity to each record in the table.
- The primary key column must only contain **unique** values.
- A table can have only **one** primary key.
- A primary key **cannot have NULL** values.

Example: Implementing Primary Key Constraint

Step 1: Create Database (if not already created)

```
CREATE DATABASE telusko_db;  
USE telusko_db;
```

Step 2: Create **employee** Table with Primary Key

```
CREATE TABLE employee (  
    id INT,  
    emp_name VARCHAR(40),  
    emp_age INT,  
    emp_city VARCHAR(40),  
    PRIMARY KEY (id)  
);
```

Here, the **id** column is defined as the **Primary Key**, ensuring unique and non-null values.

Step 3: Insert Data into **employee** Table

```
INSERT INTO employee (id, emp_name, emp_age, emp_city)  
VALUES (3, 'Rohan', 15, 'Bengaluru'),  
       (2, 'Rohan', 15, 'Bengaluru'),  
       (4, 'Rohan', 15, 'Bengaluru');
```

Step 4: Attempting to Insert Duplicate Values

If we try to execute the same **INSERT** query again, it will fail because the **id** column is a Primary Key and must contain unique values.

```
INSERT INTO employee (id, emp_name, emp_age, emp_city)
VALUES (3, 'Rohan', 15, 'Bengaluru'); -- This will cause an error
```

Error Message:

```
ERROR 1062 (23000): Duplicate entry '3' for key 'PRIMARY'
```

This error occurs because `id = 3` already exists in the table.

SQL Constraints

SQL constraints are rules applied to table columns to enforce data integrity and maintain accurate and reliable data in the database.

Types of SQL Constraints:

1. **NOT NULL** → Ensures that a column cannot have a NULL (empty) value.
2. **CHECK** → Enforces a condition on the values allowed in a column.
3. **DEFAULT** → Sets a default value for a column if no value is provided during an **INSERT** operation.
4. **UNIQUE** → Ensures all values in a column (or combination of columns) are unique across the entire table.
5. **FOREIGN KEY** → Establishes a link between data in two tables.
6. **PRIMARY KEY** → Uniquely identifies each record in a table.

Note: A table can have only **one** primary key, but it can have **multiple unique columns**.

Example: Implementing SQL Constraints

Step 1: Create Table with Constraints

```
CREATE TABLE studentInfo (  
    id INT NOT NULL,  
    name VARCHAR(40) NOT NULL,  
    age INT NOT NULL CHECK (age >= 18),  
    gender VARCHAR(10) NOT NULL,  
    phone VARCHAR(10) NOT NULL UNIQUE,  
    city VARCHAR(20) DEFAULT 'Bengaluru',  
    PRIMARY KEY (id)  
);
```

Step 2: Insert Data into **studentInfo** Table

```
-- Inserting a valid record  
INSERT INTO studentInfo (id, name, age, gender, phone, city)  
VALUES (1, 'Rahul', 19, 'M', '423146', 'Bengaluru');  
  
-- Using DEFAULT constraint (city will automatically be 'Bengaluru')  
INSERT INTO studentInfo (id, name, age, gender, phone)  
VALUES (2, 'Rohit', 20, 'M', '425146');  
  
-- Violating UNIQUE constraint (phone number must be unique)
```

```
INSERT INTO studentInfo (id, name, age, gender, phone, city)
VALUES (3, 'Sushil', 22, 'M', '425146'); -- This will cause an error

-- Using CHECK constraint (age must be 18 or above)
INSERT INTO studentInfo (id, name, age, gender, phone, city)
VALUES (4, 'Harsh', 18, 'M', '424156');
```

Error Handling:

1. UNIQUE Constraint Violation:

```
ERROR 1062 (23000): Duplicate entry '425146' for key 'phone'
```

Solution: Ensure that phone numbers are unique before inserting.

2. CHECK Constraint Violation (if age < 18):

```
ERROR 3819 (HY000): Check constraint 'studentInfo_chk_1' is violated.
```

Solution: Insert only records where `age >= 18`.

SQL constraints help maintain **data integrity** and **consistency** by enforcing rules on columns. Using constraints effectively prevents invalid data entry and ensures database reliability.

SELECT Command: Retrieving Data & Using Aliases

SQL provides the `SELECT` statement to retrieve data from one or more tables.

Retrieving Data from Tables

Select All Columns

To fetch all columns from a table, use:

```
SELECT * FROM table_name;
```

Example:

```
SELECT * FROM employees;
```

This query retrieves all columns from the `employees` table.

Select Specific Columns

To fetch specific columns, specify their names:

```
SELECT column1, column2 FROM table_name;
```

Example:

```
SELECT id, emp_name, emp_age, emp_city FROM employee;
```

This query retrieves only the `id`, `emp_name`, `emp_age`, and `emp_city` columns from the `employee` table.

Using Aliases in SQL

- Aliases are temporary names assigned to columns or tables in a query to improve readability.
- Aliases can be assigned using the `AS` keyword.
- If an alias consists of multiple words, double quotes (") or backticks (`) are used.

Using Column Aliases

```
SELECT column_name AS alias_name FROM table_name;
```

Example:

```
SELECT id AS "Emp ID",  
       emp_name AS "Employee Name",  
       emp_age AS Age,  
       emp_city AS City  
FROM employee;
```

Explanation:

- `id` is displayed as **Emp ID**
- `emp_name` is displayed as **Employee Name**
- `emp_age` is displayed as **Age**
- `emp_city` is displayed as **City**

Using Table Aliases

Table aliases are useful when working with **joins** or **self-joins**.

```
SELECT t1.column_name  
FROM table_name AS t1;
```

Example:

```
SELECT e.id, e.emp_name  
FROM employee AS e;
```

Explanation:

- The `employee` table is temporarily renamed as `e`, so column names can be prefixed with `e`.

Conclusion

Using **SELECT** effectively helps retrieve the necessary data, while **aliases** improve query readability and efficiency, especially in complex queries.

SELECT Command: SELECT with WHERE Clause

The **WHERE clause** is used in SQL to filter records based on specified conditions. This allows users to retrieve only the necessary data instead of fetching the entire table.

Syntax of WHERE Clause

```
SELECT column1, column2 FROM table_name WHERE condition;
```

Example Queries

Select with a Specific Condition

```
SELECT * FROM personalInfo WHERE id = 1;
```

Retrieves all columns for the record where **id** is 1.

Filtering by City

```
SELECT * FROM personalInfo WHERE city = 'Pune';
```

Retrieves all records where the city is **Pune**.

Using Comparison Operators

```
SELECT * FROM personalInfo WHERE age >= 18;
```

Retrieves all records where **age** is **18 or older**.

Using WHERE with Aliases

```
SELECT id AS "Emp ID", emp_name AS "Employee Name", emp_age AS Age, emp_c  
FROM employee WHERE id = 3;
```

Explanation:

- Retrieves employee details **only** where `id = 3`.
- Uses **aliases** to rename columns for readability.

Common Operators in **WHERE** Clause

Operator	Description
<code>=</code>	Equal to
<code>!=</code> or <code><></code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to
<code>AND</code>	Combines multiple conditions (both must be true)
<code>OR</code>	Combines multiple conditions (either can be true)

Example of Multiple Conditions

```
SELECT * FROM employee WHERE id = 2 AND age > 18;
```

It will retrieve records where 'id' is 2 and 'age' is greater than 18.

Conclusion

The **WHERE** clause is essential for fetching specific records based on conditions. Using it effectively helps improve query efficiency and data retrieval accuracy.

SQL: AND, OR, NOT Operators

Theory

- **AND Operator:** Combines two or more conditions and returns rows only when all conditions are true.
- **OR Operator:** Combines two or more conditions and returns rows when any one of the conditions is true.
- **NOT Operator:** Used to negate a condition; it returns rows where the condition is false.

Commutative, Associative, and Distributive Properties

Commutative Property

- The **commutative property** means that the order of the conditions does not affect the result.
- Applicable for both **AND** and **OR** operators.

Example (Commutative):

```
SELECT * FROM studentinfo WHERE age=18 OR city='Pune';  
SELECT * FROM studentinfo WHERE city='Pune' OR age=18;
```

Result: Both queries produce the same output because the OR operator is commutative.

Associative Property

- The **associative property** means that the grouping of conditions does not affect the result.
- Applicable for both **AND** and **OR** operators.

Example (Associative):

```
SELECT * FROM studentinfo WHERE (age=18 OR city='Pune') OR city='Mumbai';  
SELECT * FROM studentinfo WHERE age=18 OR (city='Pune' OR city='Mumbai');
```

Result: Both queries produce the same output because the OR operator is associative.

Distributive Property

- The **distributive property** shows how **AND** and **OR** work together.
- A condition with **AND** and **OR** can be distributed as:
 - $(A \text{ AND } (B \text{ OR } C))$ is equivalent to $((A \text{ AND } B) \text{ OR } (A \text{ AND } C))$.
 - $(A \text{ OR } (B \text{ AND } C))$ is equivalent to $((A \text{ OR } B) \text{ AND } (A \text{ OR } C))$.

Example (Distributive):

```
SELECT * FROM studentinfo WHERE age=18 AND (city='Pune' OR city='Mumbai')
SELECT * FROM studentinfo WHERE (age=18 AND city='Pune') OR (age=18 AND c
```

Result: Both queries produce the same output because the AND and OR operators distribute correctly.

Examples and Outputs

1. Using OR Operator

Query:

```
SELECT * FROM studentinfo WHERE age=18 OR age=20;
```

Output:

S.ID	Name	Age	City
1	John	18	Bengaluru
3	Rahul	20	Pune
5	Neha	18	Mumbai

Explanation: This query retrieves rows where the age is either 18 or 20.

2. Using AND Operator

Query:

```
SELECT * FROM studentinfo WHERE age=18 AND city='Pune';
```

Output:

S.ID	Name	Age	City
3	Rahul	18	Pune

Explanation: This query retrieves rows where both conditions (age is 18 and city is Pune) are true.

3. Combining AND and OR Operators

Query:

```
SELECT * FROM studentinfo WHERE city='Bengaluru' OR city='London' AND age
```

Output:

S.ID	Name	Age	City
2	Rohit	19	London
4	Priya	22	Bengaluru

Explanation: The query retrieves rows where the city is Bengaluru or where both city is London and age is 19.

4. Using NOT Operator

Query:

```
SELECT * FROM studentinfo WHERE NOT (city='Bengaluru' OR city='London') /
```

Output:

S.ID	Name	Age	City
6	Akash	19	Pune

Explanation: This query retrieves rows where the city is neither Bengaluru nor London, and the age is 19.

Summary

- Use **AND** to filter rows meeting all conditions.
 - Use **OR** to filter rows meeting any of the conditions.
 - Use **NOT** to exclude rows matching a condition.
 - **Commutative Property:** The order of conditions does not matter for **AND** and **OR**.
 - **Associative Property:** The grouping of conditions does not matter for **AND** and **OR**.
 - **Distributive Property:** Conditions with **AND** and **OR** can be distributed equivalently for clarity and efficiency.
-

IN Operator in SQL

Definition

The **IN** operator in SQL is used to filter records based on whether a column's value matches any value in a specified list. It simplifies writing multiple **OR** conditions in a query.

Syntax

```
SELECT column1, column2, ...
FROM tableName
WHERE columnName IN (value1, value2, ...);
```

Example 1: Using IN Operator

Table: Student

sid	name	age	gender
1	Rohan	19	M
2	Rohit	20	M
3	Rahul	18	M
4	Raksha	22	F

Query

```
SELECT *
FROM studentinfo
WHERE age IN (18, 19);
```

Output

id	name	age	gender	city
1	Rohan	19	M	Bengaluru
3	Rahul	18	M	London
4	Varsha	18	F	Mumbai

Example 2: Using NOT IN Operator

The **NOT IN** operator excludes rows where a column's value matches any value in a specified list.

Query

```
SELECT *  
FROM studentinfo  
WHERE id NOT IN (1, 2, 3);
```

Output

id	name	age	gender	city
4	Varsha	18	F	Mumbai
5	Sushil	20	M	Bengaluru
6	Raksha	21	F	Pune

Notes

- The **IN** operator is a concise way to filter specific values, as opposed to writing multiple **OR** conditions.
- The **NOT IN** operator is useful when excluding specific values from the results.

BETWEEN and NOT BETWEEN Operator in SQL

Definition

The **BETWEEN** operator in SQL is used to filter records where a column's value lies within a specified range, inclusive of the range's boundaries.

Syntax

```
SELECT column1, column2, ...
FROM tableName
WHERE columnName BETWEEN value1 AND value2;
```

Example 1: Using BETWEEN Operator

Table: Student

sid	name	age	gender
1	Rohan	19	M
2	Rohit	20	M
3	Rahul	18	M
4	Raksha	22	F

Query

```
SELECT *
FROM studentinfo
WHERE age BETWEEN 18 AND 20;
```

Output

id	name	age	gender	city
1	Rohan	18	M	Bengaluru
2	Rawal	19	M	London
3	Rohit	20	M	Pune

NOT BETWEEN Operator in SQL

Definition

The **NOT BETWEEN** operator excludes records where a column's value lies within the specified range.

Syntax

```
SELECT column1, column2, ...  
FROM tableName  
WHERE columnName NOT BETWEEN value1 AND value2;
```

Example 2: Using NOT BETWEEN Operator

Query

```
SELECT *  
FROM studentinfo  
WHERE id NOT BETWEEN 1 AND 4;
```

Output

id	name	age	gender	city
5	Sushil	20	M	Bengaluru
6	Raksha	21	F	Pune

Notes

- The **BETWEEN** operator is inclusive of the boundary values.
- The **NOT BETWEEN** operator helps exclude rows that fall within a specific range.

SELECT with ORDER BY in SQL

Definition

The **ORDER BY** clause in SQL is used to sort the result set of a query by one or more columns. It can sort the data in ascending (default) or descending order.

Syntax

```
SELECT columns
FROM tableName
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC], ...;
```

Key Points

- **ASC**: Sorts the data in ascending order (default).
- **DESC**: Sorts the data in descending order.
- Can be used with one or multiple columns.
- Can be combined with the **WHERE** clause to filter and sort results.

Example 1: Basic ORDER BY

Query

```
SELECT *
FROM studentinfo
ORDER BY name;
```

Output

id	name	age	gender	city
5	Anjali	21	F	Pune
3	Mehtab	20	M	Pune
2	Rawal	19	M	London
1	Rohan	18	M	Bengaluru
4	Varsha	18	F	Mumbai

Example 2: ORDER BY with DESC

Query

```
SELECT *  
FROM studentinfo  
ORDER BY age DESC;
```

Output

id	name	age	gender	city
5	Sushil	22	F	Bengaluru
6	Anjali	21	F	Pune
3	Mehtab	20	M	Pune
2	Rawal	19	M	London
1	Rohan	18	M	Bengaluru

Example 3: ORDER BY with WHERE Clause

Query

```
SELECT *  
FROM studentinfo  
WHERE age = 18  
ORDER BY name;
```

Output

id	name	age	gender	city
1	Rohan	18	M	Bengaluru
4	Varsha	18	F	Mumbai

Notes

- By default, **ORDER BY** sorts in ascending order.
- To sort by multiple columns, specify them in the order you want, separated by commas.
- When used with **WHERE**, it first filters the records and then sorts them.
- Sorting with **DESC** is helpful when the latest or largest values are needed first.

DISTINCT Operator in SQL

Definition

The **DISTINCT** operator in SQL is used to remove duplicate records from the result set. It ensures that only unique values are returned for the specified column(s).

Syntax

```
SELECT DISTINCT column1, column2, ...
FROM tableName;
```

Key Points

- It applies to the selected columns and filters out duplicate rows.
- Can be combined with the **ORDER BY** clause to sort the unique results.

Example 1: Basic DISTINCT Usage

Query

```
SELECT DISTINCT city
FROM studentinfo;
```

Table: Student

sid	name	age	gender	city
1	Rohan	19	M	Pune
2	Anjali	18	F	Bengaluru
3	Mehtab	18	M	Pune
4	Raksha	22	F	Bengaluru

Output

city
Pune
Bengaluru

Example 2: DISTINCT with ORDER BY

Query

```
SELECT DISTINCT city  
FROM studentinfo  
ORDER BY city;
```

Output

city
Bengaluru
Pune

Notes

- The **DISTINCT** operator is useful for analyzing unique values in a column.
- When combined with **ORDER BY**, it sorts the unique values in ascending (default) or descending order.
- Applies to the result set after the SELECT clause is processed.

Updating Records in SQL

Definition

The **UPDATE** statement in SQL is used to modify existing records in a table. It allows changes to one or more columns for specific rows or the entire table, based on a specified condition.

Syntax

```
UPDATE tableName
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Key Points

- The **SET** clause specifies the columns and their new values.
- The **WHERE** clause specifies which rows should be updated.
- Without a **WHERE** clause, all rows in the table will be updated.

Example 1: Updating a Single Column

Query

```
UPDATE employee
SET emp_age = 28
WHERE id = 1;
```

Before Update

id	emp_name	emp_age	emp_city
1	Rohan	26	Bengaluru
2	Rohit	24	Pune
3	Ravi	28	Mumbai

After Update

id	emp_name	emp_age	emp_city
1	Rohan	28	Bengaluru
2	Rohit	24	Pune
3	Ravi	28	Mumbai

Example 2: Updating Multiple Columns

Query

```
UPDATE employee
SET emp_age = 28, emp_city = 'Delhi'
WHERE id = 1;
```

After Update

id	emp_name	emp_age	emp_city
1	Rohan	28	Delhi
2	Rohit	24	Pune
3	Ravi	28	Mumbai

Caution

- **Always use the WHERE clause:** Omitting the **WHERE** clause will update all rows in the table.
- **Backup your data:** Before performing updates, ensure that data is backed up to avoid accidental loss.
- **Test the query:** Test your **UPDATE** statement on a small sample or in a test environment.
- **Verify changes:** After the update, run a **SELECT** query to verify that the intended changes were applied correctly.

Notes

- The **UPDATE** statement is powerful for maintaining and modifying data.
- Combine with conditions carefully to avoid unintended updates.
- Changes can be made to one or multiple columns simultaneously.

Deleting Records in SQL

Definition

The **DELETE** statement in SQL is used to remove one or more rows from a table. It allows you to delete specific records based on a condition or all rows when no condition is specified.

Syntax

```
DELETE FROM tableName;
```

Deletes all records from the table.

```
DELETE FROM tableName WHERE condition;
```

Deletes specific records based on the condition.

Key Points

- If no **WHERE** clause is provided, all rows in the table will be deleted.
- To delete only specific rows, include a **WHERE** clause with the appropriate condition.
- **DELETE** does not delete the table structure or its schema, only the data within the table.

Example 1: Delete with Condition

Query

```
DELETE FROM employee WHERE id = 2;
```

Before Deletion

id	emp_name	emp_age	emp_city
1	Rohan	15	Bengaluru
2	Rohit	15	Bengaluru
3	Ravi	15	Bengaluru

After Deletion

id	emp_name	emp_age	emp_city
1	Rohan	15	Bengaluru
3	Ravi	15	Bengaluru

Example 2: Delete All Records

Query

```
DELETE FROM employee;
```

Result

All rows in the table are removed, but the table structure remains intact.

Caution

- **Always use the WHERE clause:** Omitting the **WHERE** clause will delete all rows in the table.
- **Backup data:** Ensure you have a backup before performing a delete operation to prevent accidental data loss.
- **Test on a subset:** Run your delete query on a smaller subset of data or a test environment before applying it to the full dataset.

Notes

- Use **DELETE** for removing specific records while preserving the table structure.
- Use **TRUNCATE** if you want to delete all records faster without preserving individual delete logs (but it does not allow conditional deletion).
- Always review your delete queries carefully to avoid unintended data loss.

COMMIT and ROLLBACK in SQL

Definition

COMMIT and **ROLLBACK** are SQL commands used to manage transactions:

- **COMMIT**: Saves all the changes made during the current transaction permanently to the database.
- **ROLLBACK**: Reverts all the changes made during the current transaction back to the state before the transaction started.

Syntax

COMMIT

```
COMMIT;
```

ROLLBACK

```
ROLLBACK;
```

Key Points

- A **transaction** is a sequence of operations performed as a single logical unit of work.
- Transactions begin automatically when a DML (Data Manipulation Language) statement like `INSERT`, `UPDATE`, or `DELETE` is executed.
- Changes made during a transaction are temporary until a **COMMIT** or **ROLLBACK** is issued.
- **COMMIT** makes the changes permanent, while **ROLLBACK** undoes the changes.

Example 1: Using COMMIT

Scenario

You want to add a new employee to the `employee` table.

Query

```
BEGIN;  
INSERT INTO employee (id, emp_name, emp_age, emp_city)  
VALUES (4, 'Anjali', 22, 'Pune');  
COMMIT;
```


Before Transaction

id	emp_name	emp_age	emp_city
1	Rohan	28	Delhi
2	Rohit	24	Bengaluru
3	Ravi	28	Mumbai

After COMMIT

id	emp_name	emp_age	emp_city
1	Rohan	28	Delhi
2	Rohit	24	Bengaluru
3	Ravi	28	Mumbai
4	Anjali	22	Pune

Example 2: Using ROLLBACK

Scenario

You mistakenly update the city of an employee and want to undo the change.

Query

```
BEGIN;  
UPDATE employee  
SET emp_city = 'Hyderabad'  
WHERE id = 3;  
ROLLBACK;
```

Before Transaction

id	emp_name	emp_age	emp_city
1	Rohan	28	Delhi
2	Rohit	24	Bengaluru
3	Ravi	28	Mumbai

After ROLLBACK

id	emp_name	emp_age	emp_city
1	Rohan	28	Delhi
2	Rohit	24	Bengaluru
3	Ravi	28	Mumbai

When to Use COMMIT and ROLLBACK

- **COMMIT:**
 - After completing a sequence of operations successfully.
 - To ensure changes are saved permanently.
 - Use when you're confident that no errors exist in the transaction.
- **ROLLBACK:**
 - To undo unintended changes or errors during a transaction.
 - To ensure the database remains in a consistent state if an operation fails.

Why Use COMMIT and ROLLBACK

- To maintain **data integrity** and prevent accidental or incomplete changes.
- To give flexibility in testing and undoing operations during a transaction.
- To ensure consistency in multi-step processes where all operations must succeed or none should be applied.

Notes for Beginners

- Always use **COMMIT** or **ROLLBACK** explicitly if working in transactional mode.
- If auto-commit mode is on (default in some databases), each DML statement is automatically committed.
- Use **BEGIN** to start a transaction explicitly in environments where auto-commit is enabled.
- Carefully test queries in a transaction before committing to avoid unwanted changes.

Primary Key and Foreign Key Constraints in SQL

Definition

Primary Key

- A **Primary Key** is a column (or combination of columns) in a table that uniquely identifies each row.
- A table can only have one primary key.

Foreign Key

- A **Foreign Key** is a column (or combination of columns) in one table that establishes a link to the **Primary Key** of another table.
- It is used to maintain referential integrity between tables.

Key Points

Primary Key

- Must contain unique values.
- Cannot have NULL values.

Foreign Key

- References the primary key in another table.
- The values in the foreign key column must match the values in the referenced primary key column or be NULL.

Syntax

Creating a Primary Key

```
CREATE TABLE tableName (  
    column1 DataType,  
    column2 DataType,  
    PRIMARY KEY (column1)  
);
```

Creating a Foreign Key

```
CREATE TABLE tableName (  
    column1 DataType,
```

```
column2 DataType,  
FOREIGN KEY (column2) REFERENCES otherTable(column1)  
);
```

Example 1: Primary Key

Query

```
CREATE TABLE city (  
    id INT AUTO_INCREMENT,  
    city_name VARCHAR(20),  
    PRIMARY KEY (id)  
);  
  
INSERT INTO city (city_name)  
VALUES ('Pune'), ('Bengaluru'), ('Mumbai'), ('London');
```

Output

id	city_name
1	Pune
2	Bengaluru
3	Mumbai
4	London

Example 2: Foreign Key

Query

```
CREATE TABLE student (  
    id INT AUTO_INCREMENT,  
    name VARCHAR(50),  
    age INT,  
    city INT,  
    PRIMARY KEY (id),  
    FOREIGN KEY (city) REFERENCES city(id)  
);  
  
INSERT INTO student (name, age, city)  
VALUES ('Nikhil', 20, 3), ('Rahul', 22, 1);
```

Output

id	name	age	city
1	Nikhil	20	3
2	Rahul	22	1

Conditions for Primary Key

- Unique constraint on the column(s).
- Column(s) cannot have NULL values.

Conditions for Foreign Key

- The referenced table must have a primary key.
- Values in the foreign key column must match values in the referenced primary key column or be NULL.

Why Use Primary and Foreign Keys

- **Primary Key** ensures data uniqueness and prevents duplicate records.
- **Foreign Key** establishes relationships between tables, enabling data consistency and referential integrity.

Notes

- A single table can have multiple foreign keys, but only one primary key.
- Violating foreign key constraints (e.g., inserting a non-existent reference) will result in an error.

INNER JOIN in SQL

Definition

The **INNER JOIN** clause in SQL is used to combine rows from two or more tables based on a related column between them. It retrieves only the records that have matching values in both tables.

Syntax

```
SELECT column1, column2, ...
FROM table1
INNER JOIN table2
ON table1.common_column = table2.common_column;
```

Key Properties

- Combines rows from two or more tables where there is a match in the related column(s).
- If no match is found, the row is excluded from the result set.
- Represents the intersection of two tables in terms of data.

Example: INNER JOIN with City and Student Tables

Tables:

City Table

id	city_name
1	Pune
2	Bengaluru
3	Mumbai
4	London

Student Table

id	name	age	city
1	Nikhil	20	3
2	Rahul	22	1
3	Meera	19	2
4	Suman	21	5

Query

```
SELECT student.name, student.age, city.city_name
FROM student
INNER JOIN city
ON student.city = city.id;
```

Output

name	age	city_name
Nikhil	20	Mumbai
Rahul	22	Pune
Meera	19	Bengaluru

Condition

- The **ON** clause specifies the condition for matching rows in both tables (e.g., `student.city = city.id`).
- Rows without a match in the other table are excluded from the result set.

Venn Diagram Explanation

- **Inner Join** corresponds to the intersection area of the two circles representing the tables.
- Only the matching data that exists in both tables is retrieved.

Notes

- Use **INNER JOIN** when you only need data that exists in both tables.
- Ensure the columns used in the **ON** clause have matching data types to avoid errors.
- For tables with no matching values, the result set will be empty.

SQL LEFT JOIN

Definition

The **LEFT JOIN** clause in SQL is used to retrieve all records from the left table (Table 1) and the matching records from the right table (Table 2). If there is no match, NULL values are returned for the columns from the right table.

Syntax

```
SELECT column1, column2, ...
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

Key Properties

- Retrieves all rows from the left table.
- Retrieves matching rows from the right table.
- If there is no match, the columns from the right table will contain NULL values.

Example: LEFT JOIN with City and Student Tables

Tables:

City Table

id	city_name
1	Pune
2	Bengaluru
3	Mumbai
4	London

Student Table

id	name	age	city
1	Nikhil	20	3
2	Rahul	22	1
3	Meera	19	2
4	Suman	21	5

Query

```
SELECT student.name, student.age, city.city_name
FROM student
LEFT JOIN city
ON student.city = city.id;
```

Output

name	age	city_name
Nikhil	20	Mumbai
Rahul	22	Pune
Meera	19	Bengaluru
Suman	21	NULL

Condition

- The **ON** clause specifies the condition for matching rows in both tables (e.g., `student.city = city.id`).
- Rows from the left table with no match in the right table will still appear in the result set, with NULL values for the right table columns.

Notes

- Use **LEFT JOIN** when you want all rows from the left table regardless of whether there is a match in the right table.
- Useful for scenarios where you want to include all data from one table while optionally matching it with another table.

SQL RIGHT JOIN

Definition

The **RIGHT JOIN** clause in SQL is used to retrieve all records from the right table (Table 2) and the matching records from the left table (Table 1). If there is no match, NULL values are returned for the columns from the left table.

Syntax

```
SELECT column1, column2, ...
FROM table1
RIGHT JOIN table2
ON table1.common_column = table2.common_column;
```

Key Properties

- Retrieves all rows from the right table.
- Retrieves matching rows from the left table.
- If there is no match, the columns from the left table will contain NULL values.

Example: RIGHT JOIN with City and Student Tables

Tables:

City Table

id	city_name
1	Pune
2	Bengaluru
3	Mumbai
4	London

Student Table

id	name	age	city
1	Nikhil	20	3
2	Rahul	22	1
3	Meera	19	2
4	Suman	21	5

Query

```
SELECT student.name, student.age, city.city_name
FROM student
RIGHT JOIN city
ON student.city = city.id;
```

Output

name	age	city_name
Rahul	22	Pune
Meera	19	Bengaluru
Nikhil	20	Mumbai
NULL	NULL	London

Condition

- The **ON** clause specifies the condition for matching rows in both tables (e.g., `student.city = city.id`).
- Rows from the right table with no match in the left table will still appear in the result set, with NULL values for the left table columns.

Notes

- Use **RIGHT JOIN** when you want all rows from the right table regardless of whether there is a match in the left table.
- Useful for scenarios where you want to include all data from one table while optionally matching it with another table.

SQL CROSS JOIN

Definition

The **CROSS JOIN** clause in SQL is used to combine every row from the first table with every row from the second table. It produces the Cartesian product of the two tables.

Syntax

```
SELECT column1, column2, ...  
FROM table1  
CROSS JOIN table2;
```

Key Properties

- Produces a Cartesian product of the two tables.
- The number of rows in the result set equals the product of the number of rows in the two tables.
- No condition is required for a CROSS JOIN.

Example: CROSS JOIN with City and Student Tables

Tables:

City Table

id	city_name
1	Pune
2	Bengaluru
3	Mumbai

Student Table

id	name	age
1	Nikhil	20
2	Rahul	22

Query

```
SELECT student.name, student.age, city.city_name
FROM student
CROSS JOIN city;
```

Output

name	age	city_name
Nikhil	20	Pune
Nikhil	20	Bengaluru
Nikhil	20	Mumbai
Rahul	22	Pune
Rahul	22	Bengaluru
Rahul	22	Mumbai

Condition

- A CROSS JOIN does not require a matching condition between the two tables.
- To filter the results, you can use a **WHERE** clause after the join.

Notes

- Use **CROSS JOIN** cautiously as it can produce a large number of rows if the tables are large.
- Useful for scenarios like generating combinations or pairing data from two unrelated tables.

SQL ALTER Command

Definition

The **ALTER** command in SQL is used to modify the structure of an existing table. It allows you to add, delete, or modify columns, as well as add or drop constraints.

Syntax

```
ALTER TABLE tableName
<modification>;
```

Use Cases with Examples and Outputs

1. Adding a Column

- Use the **ADD** clause to add a new column to a table.

```
ALTER TABLE student
ADD phone_number VARCHAR(15);
```

Before Execution:

id	name	age
1	Rahul	20
2	Meera	22

After Execution:

id	name	age	phone_number
1	Rahul	20	NULL
2	Meera	22	NULL

2. Modifying a Column

- Use the **MODIFY** clause (MySQL) or **ALTER COLUMN** (SQL Server) to change the data type or properties of an existing column.

```
ALTER TABLE student
MODIFY phone_number CHAR(10);
```

Before Execution:

id	name	age	phone_number
1	Rahul	20	NULL
2	Meera	22	NULL

After Execution:

id	name	age	phone_number
1	Rahul	20	NULL
2	Meera	22	NULL
(Note: Data type changed to CHAR(10))			

3. Renaming a Column

- Use the **RENAME COLUMN** clause (MySQL 8.0+, Oracle) to rename a column.

```
ALTER TABLE student
RENAME COLUMN phone_number TO contact_number;
```

Before Execution:

id	name	age	phone_number
1	Rahul	20	NULL
2	Meera	22	NULL

After Execution:

id	name	age	contact_number
1	Rahul	20	NULL
2	Meera	22	NULL

4. Dropping a Column

- Use the **DROP** clause to remove a column from a table.

```
ALTER TABLE student
DROP COLUMN contact_number;
```

Before Execution:

id	name	age	contact_number
1	Rahul	20	NULL
2	Meera	22	NULL

After Execution:

id	name	age
1	Rahul	20
2	Meera	22

5. Adding Constraints

- Use the **ADD CONSTRAINT** clause to add a primary key.

```
ALTER TABLE student
ADD CONSTRAINT pk_student PRIMARY KEY (id);
```

Before Execution:

id	name	age
1	Rahul	20
2	Meera	22

After Execution:

The column `id` is now the primary key of the table.

6. Dropping Constraints

- Use the **DROP CONSTRAINT** clause to remove constraints.

```
ALTER TABLE student
DROP CONSTRAINT pk_student;
```

Before Execution:

The column `id` is the primary key of the table.

After Execution:

The `id` column no longer has a primary key constraint.

7. Renaming a Table

- Use the **RENAME TO** clause to rename a table.

```
ALTER TABLE student
RENAME TO student_info;
```

Before Execution:

The table name is `student`.

After Execution:

The table name is now `student_info`.

Notes

- Changes made with **ALTER** are permanent and cannot be undone without explicit commands.
- Always test **ALTER** commands on a backup or test environment before applying to a production database.
- Not all ALTER functionalities are supported in every RDBMS. Check your database's documentation for specific syntax.

This command is essential for maintaining and evolving database schema as application requirements change.

SQL DROP and TRUNCATE Commands

Definition

DROP Command

The **DROP** command in SQL is used to permanently delete a table, its structure, and all of its data from the database.

TRUNCATE Command

The **TRUNCATE** command in SQL is used to quickly remove all rows from a table while keeping the table structure intact.

Syntax

DROP

```
DROP TABLE tableName;
```

TRUNCATE

```
TRUNCATE TABLE tableName;
```

Key Differences

Feature	DROP	TRUNCATE
Deletes data	Yes	Yes
Deletes structure	Yes	No
Transaction support	No	Limited (non-transactional in some RDBMS)
Speed	Slower (removes each row and structure)	Faster (removes data directly)
Rollback capability	No	No

Use Cases with Examples and Outputs

1. DROP Command

- Permanently deletes the table and its data.

Query:

```
DROP TABLE student;
```

Before Execution:

id	name	age
1	Rahul	20
2	Meera	22

After Execution:

The `student` table no longer exists in the database.

2. TRUNCATE Command

- Quickly removes all rows from a table.

Query:

```
TRUNCATE TABLE student;
```

Before Execution:

id	name	age
1	Rahul	20
2	Meera	22

After Execution:

id	name	age
(empty)		

The table structure remains intact but all rows are deleted.

Conditions for Use

DROP

- Use when you no longer need a table, including its structure and data.

TRUNCATE

- Use when you want to quickly clear all rows from a table but retain its structure for future use.

Cautions

- **DROP**: Once executed, the table and its data cannot be recovered unless backed up.
- **TRUNCATE**: Data removal is permanent and cannot be rolled back.
- Both commands bypass triggers and do not generate individual delete logs, making them faster but riskier.

Notes

- **TRUNCATE** is often preferred for quickly clearing data while retaining the table for reuse.
- **DROP** is useful when completely removing unused tables to clean up the database.
- Neither command should be used without careful consideration and proper backups in place.